# Pitstop Oracle — ML Model Design

Version 3 | Evaluation-first model strategy, leakage controls, and forecast contracts

## 1. Purpose and principles

This design is evaluation-first. Reproduce the current results, audit feature timing, and establish trustworthy baselines before adding model complexity.

README-reported test metrics: pole baseline 69.4%, championship leader 30.6%, logistic regression 52.8%, random forest 58.3%, and random-forest top-3 hit rate 100%. These figures have not been independently reproduced in a source audit.

- Compare models on identical races and information cutoffs.
- Keep pole sitter as a permanent post-qualifying benchmark.
- Separate pre-qualifying and post-qualifying forecasts.
- Use chronological splits; do not randomly mix future races into training.
- Do not call outputs probabilities unless calibration is evaluated.

## 2. Existing pipeline

| Stage | Described component | Responsibility |
|---|---|---|
| Ingestion | Jolpica + Open-Meteo; ingest.py | Raw race and weather inputs |
| Features | features.py | Driver-race feature table / Parquet |
| Training | train.py | Serialized model artifacts |
| Prediction | predict.py | Prediction interface |
| UI | ui.py and Streamlit pages | Forecast, scenarios, history, metrics |

README states no telemetry and no paid APIs. Do not introduce telemetry-derived features without changing the data and product scope.

## 3. Reproduce current results

- Run pipeline from a clean environment; record dependency versions and seed.
- Record exact train, validation, and test seasons/races.
- Confirm target definition: winner, top-3, or another outcome.
- Recompute all baselines and models on identical test rows.
- Save per-race predictions, not only aggregate accuracy.
- Check handling of ties, DNS, disqualifications, sprint weekends, and incomplete results.

The evaluation report should include dataset fingerprint, feature list, split boundaries, model parameters, metrics, and per-race predictions.

## 4. Feature timing and leakage

| Feature | Pre-qualifying | Post-qualifying | Audit |
|---|---|---|---|
| Historical driver form | Prior races only | Prior races only | Shift aggregates; exclude current result |
| Constructor form | Prior races only | Prior races only | As-of event snapshot |
| Championship position | Prior standings | Prior standings | No future standings |

| Feature | Pre-qualifying | Post-qualifying | Audit |
|---|---|---|---|
| Grid / pole | Unavailable | Available | Exclude from pre-quali |
| Weather | Timestamped forecast only | Timestamped forecast only | Do not use future observed weather |
| Race result / fastest lap | Unavailable | Unavailable | Never use for winner forecast |

A historical field is not automatically valid for a historical forecast. Ask whether it would have been known at the stated prediction timestamp.

## 5. Evaluation protocol

Use chronological train, validation, and untouched test periods. If data is limited, use rolling-origin evaluation across season cutoffs.

• Pre-qualifying: exclude grid and qualifying-derived information.

• Post-qualifying: grid and qualifying data may be used if timestamped.

• Optional race-day refresh: use only data available at the declared refresh time.

• Report winner accuracy, top-3 hit rate, coverage, and per-season breakdown.

• If probabilities are shown, report log loss, Brier score, and calibration.

Baselines: pole sitter (post-qualifying only), championship leader as of cutoff, and a simple historical driver/constructor strength baseline. Compare each on the same races.

## 6. Model development sequence

| Order | Work | Reason |
|---|---|---|
| 1 | Reproduce current models | Trusted reference |
| 2 | Inspect per-race errors and feature importance | Find failure modes |
| 3 | Build leakage-safe rolling features | Improve signal quality |
| 4 | Tune regularization / tree depth | Avoid overfit |
| 5 | Test gradient boosting if justified | Stronger tabular baseline |
| 6 | Calibrate on held-out validation data | Only if probabilities needed |
| 7 | Freeze version and test untouched period | Protect final evaluation |

Do not jump straight to deep learning. Race-event datasets are relatively small and complex models can overfit.

## 7. Forecast output contract

| Field | Purpose |
|---|---|
| event_id / season / round | Event identity |
| forecast_stage | Pre- or post-qualifying / declared cutoff |
| generated_at / data_as_of | Creation and source freshness |
| model_version | Exact model artifact |
| feature_snapshot_id | Exact input snapshot |
| driver_id | Stable driver identity |
| score or probability | Explicit output semantics |
| rank / warnings | Ordering and data-quality notes |

Persist forecast snapshots and per-driver outputs if historical forecast tracking is required. Overwriting each refresh prevents reliable backtesting of what users actually saw.

## 8. Scenarios, audit, and acceptance

• Document assumptions changed by each scenario.

• Show baseline and scenario side by side.

• Do not imply causal effects from a simple feature override.

• Flag scenarios outside the training distribution.

• Confirm whether model outputs are scores, rankings, or calibrated probabilities.

Source files needed for code-level review: ingest.py, features.py, train.py, predict.py, ui.py, metrics.json, run_pipeline.py, and environment/dependency files.

Acceptance: results reproduce; features have availability timestamps; no leakage; model and baselines share test events; UI displays stage, version, freshness, and output semantics. This document is based on the public README, not source-level verification.